

6. Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

CODE:

```
#include <stdio.h>
```

```
#define MAX 25
```

```
void firstFit(int blockSize[], int m, int processSize[], int n) {
```

```
    int allocation[MAX];
```

```
    int temp[MAX];
```

```
    for (int i = 0; i < m; i++)
```

```
        temp[i] = blockSize[i];
```

```
    for (int i = 0; i < n; i++)
```

```
        allocation[i] = -1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (temp[j] >= processSize[i]) {
```

```
                allocation[i] = j;
```

```
                temp[j] -= processSize[i];
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("\n--- First Fit ---\n");
```

```
    printf("Process No.\tProcess Size\tBlock No.\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
```

```

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[MAX];
    int temp[MAX];

    for (int i = 0; i < m; i++)
        temp[i] = blockSize[i];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;

        for (int j = 0; j < m; j++) {
            if (temp[j] >= processSize[i]) {
                if (bestIdx == -1 || temp[j] < temp[bestIdx]) {
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            temp[bestIdx] -= processSize[i];
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
}

```

```

for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[MAX];
    int temp[MAX];

    for (int i = 0; i < m; i++)
        temp[i] = blockSize[i];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;

        for (int j = 0; j < m; j++) {
            if (temp[j] >= processSize[i]) {
                if (worstIdx == -1 || temp[j] > temp[worstIdx]) {
                    worstIdx = j;
                }
            }
        }

        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            temp[worstIdx] -= processSize[i];
        }
    }
}

```

```

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main() {
    int blockSize[MAX], processSize[MAX];
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }

    firstFit(blockSize, m, processSize, n);
    bestFit(blockSize, m, processSize, n);
    worstFit(blockSize, m, processSize, n);
}

```

```
    return 0;
}
```

OUTPUT:

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
200 300 400 500 600
Enter number of processes: 4
Enter sizes of 4 processes:
110 120 130 140

--- First Fit ---
Process No.      Process Size      Block No.
1                110                1
2                120                2
3                130                2
4                140                3

--- Best Fit ---
Process No.      Process Size      Block No.
1                110                1
2                120                2
3                130                2
4                140                3

--- Worst Fit ---
Process No.      Process Size      Block No.
1                110                5
2                120                4
3                130                5
4                140                3

Process returned 0 (0x0)   execution time : 28.100 s
Press any key to continue.
|
```